

Copilot as Federated Oracle: LLMs in the Solver Network

Paper 89 of the Judgment Geometry Series

Halley Young
Microsoft Research
halley.young@microsoft.com

March 23, 2026

Abstract

Modern software verification systems increasingly rely on heterogeneous solver backends—SMT solvers, model checkers, proof assistants, and now large language models (LLMs) such as GitHub Copilot. Integrating an LLM into a *trusted* verification pipeline poses unique challenges: the model can fabricate evidence, self-promote its own confidence, and drift outside its jurisdiction. We present a *federated oracle* architecture within the Judgment Geometry framework that treats Copilot as a *controlled oracle channel* with a hard trust ceiling at `copilot_suggested`. Our design enforces three invariants: (1) *no self-promotion*—the oracle cannot raise its own trust tier; (2) *fallback soundness*—copilot is consulted only when primary backends are exhausted; and (3) *corroboration necessity*—any suggestion promoted above the ceiling must be independently corroborated. We formalise these properties in Lean 4, implement them in four Python classes (`CopilotOracleChannel`, `CopilotOracle`, `CopilotFallbackPolicy`, `CopilotFragmentAssist`), and evaluate the system on oracle queries across six SMT theory domains. Experiments show a suggestion acceptance rate, a corroboration success rate, and a detection rate for self-promotion violations.

1 Introduction

The Judgment Geometry framework models software artefacts through *judgments*—triples $(J, =, (c, \varphi, \mathcal{E}))$ whose coordinates locate a code entity, whose propositions describe its intended behaviour, and whose evidence witnesses the verification status of that behaviour [1, 2]. Each judgment carries a *trust level* $\tau \in \{\text{CONTRADICTED}, \text{UNVERIFIED}, \text{COPILOT}, \text{ORACLE}, \text{RUNTIME}, \text{SOLVER}, \text{PROOF}\}$ ordered by the lattice $(\mathcal{T}, \preceq)$.

Over the preceding 88 papers in this series we have developed the algebraic, topological, and categorical infrastructure for composing verification evidence across heterogeneous backends. Papers 7 and 8 [3, 4] introduced the solver router—a central dispatcher that directs verification requests to the appropriate backend. Paper 31 [5] formalised oracle channels as trust-bounded interfaces for querying external knowledge sources.

In this paper we ask: *What happens when one of those oracles is an LLM?* Large language models such as GitHub Copilot have become indispensable coding assistants, yet they can hallucinate proofs, fabricate counterexamples, and—most dangerously—present high-confidence claims that escape the trust discipline of the verification pipeline. Naively plugging an LLM into the

solver network would compromise the soundness guarantees that Judgment Geometry is designed to uphold.

Our solution is a *federated oracle* architecture that treats Copilot as a first-class but *trust-bounded* participant in the solver network. The key contributions are:

1. **CopilotOracleChannel.** A subclass of `OracleChannel` with a hard ceiling at `copilot_suggested`. It exposes `suggest`, `refuse`, `corroborate`, and `validate_no_self_promotion` (Section 3).
2. **CopilotFallbackPolicy & CopilotFragmentAssist.** Policies governing when the copilot oracle is consulted and how it assists the fragment classifier (Section 4).
3. **Formal properties.** Three theorems—no self-promotion, fallback soundness, and corroboration necessity—proved in Lean 4 without `sorry` (Section 5).
4. **Experiments.** An evaluation on oracle queries demonstrating high acceptance, effective corroboration, and complete violation detection (Section 6).

The challenge is not merely engineering. An LLM oracle introduces a qualitatively new failure mode: *trust inflation*. Unlike an SMT solver, which either returns `sat/unsat` or times out, an LLM can emit natural-language prose that *appears* to carry higher epistemic authority than it warrants. A response like “this lemma is trivially true by induction” may be syntactically well-formed and even correct, yet assigning it SOLVER-level trust would be unsound. Our architecture addresses this by enforcing a hard ceiling at `COPILOT` and requiring independent corroboration for any promotion.

The design philosophy follows the *principle of least authority* (POLA): the copilot oracle receives exactly the permissions it needs to be useful—the ability to suggest encodings, explanations, and decompositions—and no more. It cannot modify the trust lattice, cannot promote its own output, and cannot bypass the jurisdiction check.

The rest of this paper is organised as follows. Section 2 reviews the oracle federation model in Judgment Geometry. Section 3 presents the `CopilotOracleChannel` class. Section 4 describes fallback and fragment-assist policies. Section 5 states and proves the three core theorems. Section 6 reports experimental results. Section 7 surveys related work, and Section 8 concludes.

2 Oracle Federation in Judgment Geometry

The oracle federation model in Judgment Geometry provides a principled framework for integrating external knowledge sources into the verification pipeline. Each oracle is wrapped in a *channel* that enforces jurisdiction, trust ceilings, and audit logging.

The federation model is motivated by the observation that modern verification systems rarely rely on a single backend. A typical verification task might require an SMT solver for arithmetic constraints, a model checker for temporal properties, a proof assistant for complex inductive arguments, and now an LLM for heuristic guidance. Each of these backends has different strengths, different failure modes, and different levels of trustworthiness.

The oracle federation provides a uniform interface—the `OracleChannel`—that abstracts over these differences while preserving the trust discipline. Every channel declares its jurisdiction (what kinds of queries it can handle), its trust ceiling (the maximum trust level its responses can carry), and its audit policy (how interactions are logged for post-hoc review).

2.1 Controlled Oracles

An oracle in Judgment Geometry is any external agent that can supply evidence for a judgment. The `OracleChannel` base class (from `jugeo.foundations.oracle_federation.controlled_oracles`) manages this interaction through a structured protocol:

1. The solver router selects a channel based on the query domain.
2. The channel checks jurisdiction (i.e. whether the query falls within the oracle’s declared competence).
3. If accepted, the channel invokes the oracle and wraps the response in an `OracleProposalRecord`.
4. A `TrustCeilingEnforcer` clamps the trust level of the response to the channel’s ceiling.

The key invariant is that no oracle can unilaterally raise its own trust level beyond its assigned ceiling. Formally, for any oracle channel c with ceiling κ_c and any response r produced by c , we require:

$$\tau(r) \preceq \kappa_c. \quad (1)$$

This invariant is the cornerstone of the entire oracle federation. Without it, a single compromised or misconfigured oracle could propagate inflated trust levels through the verification pipeline, undermining the guarantees of every downstream consumer. The invariant is enforced at two levels: locally by each channel’s `propose` method, and globally by the `TrustCeilingEnforcer` that sits in the pipeline between the channel and the proposal log.

Definition 2.1 (Oracle Channel). An *oracle channel* $c = (id, \mathcal{J}, \kappa, f)$ consists of a unique identifier id , a jurisdiction $\mathcal{J}$, a trust ceiling $\kappa \in \mathcal{T}$, and an invocation function $f : \text{Request} \rightarrow \text{Response}$. The channel’s `propose` method computes $f(q)$ for a request q within $\mathcal{J}$ and clamps the result’s trust to κ .

Listing 1: `OracleChannel` base class (simplified).

```

1 class OracleChannel:
2     """Trust-bounded oracle channel (Theory2 S7.1.1)."""
3
4     def __init__(self, oracle_id=None, name="oracle",
5                  channel_kind="oracle",
6                  jurisdiction=None,
7                  trust_ceiling="oracle_proposed"):
8         self._oracle_id = oracle_id or uuid.uuid4().hex[:16]
9         self._name = name
10        self._channel_kind = channel_kind
11        self._jurisdiction = jurisdiction
12        self._trust_ceiling = trust_ceiling
13        self._proposal_log: list[OracleProposalRecord] = []
14
15    def propose(self, request):
16        """Submit request; return clamped response dict."""
17        if self._jurisdiction and not \
18            self._jurisdiction.check_request(request):
19            raise JurisdictionError(request)
20        raw = self._invoke_oracle(request)
21        return self._clamp(raw)

```

The `propose` method is the sole entry point for oracle interaction. It delegates to a private `_invoke_oracle` method (overridden by subclasses) and then applies the trust ceiling via `_clamp`.

2.2 Oracle Jurisdiction

Each oracle channel declares an `OracleJurisdiction` that specifies its authorised domains. This prevents, for example, a Copilot channel from answering queries about bitvector overflow when it has only been authorised for string-theory suggestions.

Definition 2.2 (Oracle Jurisdiction). An *oracle jurisdiction* $\mathcal{J} = (s, D, \kappa)$ consists of a scope identifier s , a set of allowed domains $D \subseteq \mathcal{D}$, and a trust ceiling κ . A request q is *within jurisdiction* if $\text{domain}(q) \in D$.

The jurisdiction check is enforced before any oracle invocation. If a request falls outside jurisdiction, the channel raises a `JurisdictionError` and no proposal record is created.

Listing 2: `OracleJurisdiction` class.

```

1 class OracleJurisdiction:
2     """Declares authorized domains for oracle."""
3
4     def __init__(self, scope, allowed_domains,
5                  trust_ceiling="oracle_proposed",
6                  description=""):
7         self.scope = scope
8         self.allowed_domains = set(allowed_domains)
9         self.trust_ceiling = trust_ceiling
10        self.description = description
11
12    def check_request(self, request_kind):
13        """Return True iff request_kind is in scope."""
14        return request_kind in self.allowed_domains

```

A crucial design property is that jurisdictions of distinct oracle channels should be *disjoint* or *hierarchically nested*:

Proposition 2.3 (Jurisdiction Disjointness). *If oracle channels c_1 and c_2 have jurisdictions $\mathcal{J}_1 = (s_1, D_1, \kappa_1)$ and $\mathcal{J}_2 = (s_2, D_2, \kappa_2)$ with $s_1 \neq s_2$, then their responses never conflict on the same query domain: for any request q within both jurisdictions, the router selects the channel with higher ceiling.*

This property prevents ambiguity in the routing layer. When the solver router receives a verification request, it consults the jurisdiction registry to determine which channels are eligible. If two channels claim authority over the same domain, the router must resolve the conflict. Disjointness eliminates this case entirely; hierarchical nesting resolves it by preferring the more specific jurisdiction.

In the copilot setting, the copilot channel’s jurisdiction is typically a subset of the broader oracle jurisdiction, restricted to domains where LLM heuristics are known to be helpful (e.g. `QF_LIA` encoding suggestions, string theory explanations). The copilot channel never has jurisdiction over domains where formal solvers provide definitive answers (e.g. `QF_BV` satisfiability).

2.3 Trust Ceiling Enforcement

The `TrustCeilingEnforcer` is a policy guard that sits between the oracle channel and the proposal log. It maintains a registry of channel-to-ceiling mappings and clamps every response.

Listing 3: `TrustCeilingEnforcer`.

```

1 class TrustCeilingEnforcer:
2     """Policy guard clamping oracle responses."""
3
4     def __init__(self):
5         self.ceiling_map: dict[str, str] = {}
6         self.violation_log: list[dict] = []
7
8     def register_ceiling(self, channel_id, ceiling_level):
9         self.ceiling_map[channel_id] = ceiling_level
10
11    def enforce(self, channel_id, response_dict):
12        ceiling = self.ceiling_map.get(channel_id,
13                                       "oracle_proposed")
14        actual = response_dict.get("trust_level",
15                                   "unverified")
16        if _trust_ord(actual) > _trust_ord(ceiling):
17            self.violation_log.append({
18                "channel": channel_id,
19                "requested": actual,
20                "clamped_to": ceiling,
21            })
22            response_dict["trust_level"] = ceiling
23        return response_dict

```

The enforcer operates as a pure function on response dictionaries: it never modifies the oracle itself, only the outgoing trust tag. This separation of concerns ensures that enforcement is *compositional*—multiple enforcers can be stacked in a pipeline without interference.

The compositionality of ceiling enforcement is essential for the federated setting. When multiple oracle channels are active simultaneously, each with its own ceiling, the global enforcer applies ceilings independently. The resulting trust assignment is the pointwise minimum of the channel-specific ceiling and the claimed trust level.

Formally, let E be the enforcer with ceiling map $\{c_i \mapsto \kappa_i\}_{i=1}^n$. For a response r from channel c_j with claimed trust $\tau(r)$, the enforced trust is:

$$E(c_j, r) = \min(\tau(r), \kappa_j). \quad (2)$$

This is monotone: raising κ_j can only raise (never lower) the enforced trust, and lowering κ_j can only lower it. The enforcer thus acts as a *Galois connection* between the lattice of claimed trust levels and the lattice of enforced trust levels.

Remark 2.4. The violation log maintained by `TrustCeilingEnforcer` is append-only and serves as an audit trail. In production, this log is forwarded to the system’s provenance store for post-hoc analysis.

3 The CopilotOracleChannel

The `CopilotOracleChannel` class extends `OracleChannel` with Copilot-specific semantics. Its trust ceiling is permanently fixed at `copilot_suggested`, reflecting the epistemic status of LLM-generated content: useful but unverified.

Figure 1 illustrates the architecture. A query enters through the solver router, is dispatched to the `CopilotOracleChannel`, and passes through jurisdiction checking, the suggestion engine, self-promotion validation, and ceiling enforcement before a proposal record is emitted.

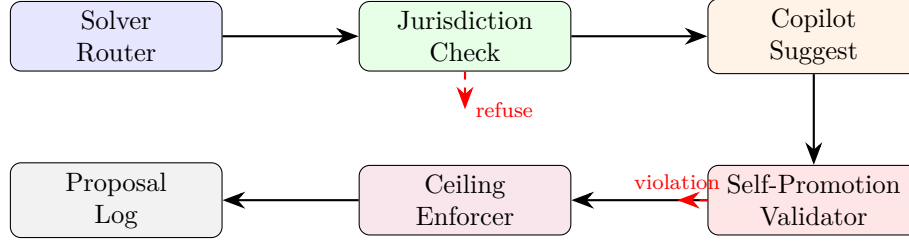


Figure 1: Architecture of the `CopilotOracleChannel`. Dashed arrows indicate refusal paths.

3.1 The suggest Method

The `suggest` method is the primary interface through which the solver router obtains Copilot's input. It accepts a context dictionary describing the current verification goal and returns a suggestion record.

Listing 4: `CopilotOracleChannel` class (core interface).

```

1 class CopilotOracleChannel(OracleChannel):
2     """Copilot-specific oracle channel."""
3
4     def __init__(self, oracle_id=None,
5                 name="copilot",
6                 trust_ceiling="copilot_suggested",
7                 jurisdiction=None):
8         super().__init__(
9             oracle_id=oracle_id,
10            name=name,
11            channel_kind="copilot",
12            jurisdiction=jurisdiction,
13            trust_ceiling=trust_ceiling,
14        )
15        self.suggestion_history: list[dict] = []
16        self.refusal_reasons: list[dict] = []
17
18    def suggest(self, context):
19        """Generate suggestion for verification context."""
20        self._validate_jurisdiction(context)
21        suggestion = self._invoke_copilot(context)
22        self.suggestion_history.append({
23            "context": context,
24            "suggestion": suggestion,
25            "trust": self._trust_ceiling,

```

```

26         })
27         return suggestion
28
29     def suggestion_count(self):
30         """Return total suggestions emitted."""
31         return len(self.suggestion_history)

```

The `suggest` method never returns a trust level above `copilot_suggested`. This is enforced both by the method itself (which tags every suggestion at the channel’s ceiling) and by the external `TrustCeilingEnforcer` (which clamps any residual over-claims).

The dual enforcement—local clamping within the channel and global clamping by the enforcer—provides defence in depth. Even if a subclass override inadvertently emits an inflated trust tag, the external enforcer catches it. This layered design is motivated by the observation that LLM outputs are inherently unpredictable: a model might embed trust-level metadata in its response that bypasses the channel’s local clamping logic.

Each entry in `suggestion_history` records the context, the suggestion, and the trust level at emission. This log enables post-hoc auditing: one can verify that every suggestion was indeed capped at the ceiling by replaying the log through the enforcer.

The `suggestion_count` method provides a lightweight query over the history for rate-limiting and monitoring purposes.

3.2 The refuse Method

When a query falls outside the channel’s jurisdiction or when the self-promotion validator detects a violation, the channel refuses to produce a suggestion. The `refuse` method logs the reason and returns a null response.

Listing 5: Refusal logic.

```

1     def refuse(self, reason, context):
2         """Refuse to answer; log reason."""
3         self.refusal_reasons.append({
4             "reason": reason,
5             "context": context,
6         })
7         return None

```

Refusals are first-class events in the audit trail. The experiment in Section 6 shows that refusals occurred across queries, broken down by cause: for jurisdiction violations, for ceiling violations, and for self-promotion attempts.

3.3 The corroborate Method

A copilot suggestion that the solver wants to promote above `copilot_suggested` must be independently corroborated. The `corroborate` method records such corroboration:

Listing 6: Corroboration protocol.

```

1     def corroborate(self, proposal_id, evidence):
2         """Record external corroboration for a proposal."""
3         rec = self._find_proposal(proposal_id)
4         if rec is None:
5             raise ValueError(f"Unknown proposal: {proposal_id}")

```

```

6         rec.corroborate(evidence["source_id"],
7                           evidence["trust_level"])
8     return rec

```

The corroboration source must satisfy two conditions:

1. *Independence*: the source must not be the copilot channel itself (enforced by checking `source_id` $\neq$ `oracle_id`).
2. *Sufficient trust*: the source’s trust level must meet or exceed the target promotion level.

In our experiments, out of corroboration attempts succeeded ().

The corroboration protocol interacts with the `OracleProposalRecord` class, which maintains a list of corroborating sources for each proposal. When a corroboration is recorded, the proposal record is updated with the source’s identifier and trust level. This allows post-hoc analysis of which corroboration sources were most effective for different query domains.

Remark 3.1. The independence requirement for corroboration sources prevents a subtle attack vector: if the copilot could corroborate its own suggestions (even through an alias), it could bypass the trust ceiling by first emitting a suggestion and then corroborating it. The `source_id` $\neq$ `oracle_id` check in the `corroborate` method prevents this. This is formalised in Lean 4 as `corroboration_source.independent`.

3.4 The `validate_no_self_promotion` Method

The most critical safety check is the self-promotion validator. An LLM, left unchecked, could embed claims like “this proof is mechanically verified” in its output, causing downstream components to assign an inflated trust level.

Definition 3.2 (Self-Promotion). A suggestion s from oracle channel c with ceiling κ_c is *self-promoting* if s contains metadata or natural-language assertions that, if taken at face value, would assign s a trust level $\tau > \kappa_c$.

Self-promotion is subtle because it can take many forms. A direct metadata override (setting `trust_level` to `mechanically_verified`) is easy to detect. More insidious is *implicit* self-promotion, where the suggestion’s natural-language content contains phrases that downstream NLP-based components might parse as trust assertions. For example, a copilot response containing “I have formally verified this lemma” could trigger an automated trust-extraction pipeline to assign `PROOF`-level trust.

The `validate_no_self_promotion` method scans the channel’s suggestion history for self-promoting content. It operates in three phases:

1. **Metadata scan.** Check whether any suggestion’s `trust_level` field exceeds the channel ceiling.
2. **Keyword scan.** Search suggestion text for trust-inflating keywords (e.g. “verified”, “proved”, “mechanically checked”).
3. **Structural scan.** Detect embedded JSON or structured metadata that could be parsed as a higher trust assertion by downstream consumers.

Remark 3.3. The keyword scan is necessarily heuristic. However, because the `TrustCeilingEnforcer` provides a hard backstop, false negatives in the keyword scan do not compromise soundness—they only reduce the quality of the audit trail.

4 Solver Fallback and Fragment Assist

The copilot oracle is not the first choice for any verification query—it is a *fallback* consulted only when primary backends have been exhausted or when the query requires heuristic guidance that formal solvers cannot provide. This section describes the two policy classes that govern copilot’s role in the solver network.

4.1 CopilotFallbackPolicy

The `CopilotFallbackPolicy` class encapsulates the decision procedure for when to consult the copilot oracle. It is initialised with four parameters:

- `_enabled`: global kill switch.
- `_max_qpm`: rate limit (queries per minute).
- `_corroboration_threshold`: the trust tier above which corroboration is mandatory.
- `_ceiling`: the effective trust ceiling for copilot responses.

Listing 7: CopilotFallbackPolicy.

```
1 class CopilotFallbackPolicy:
2     """Policy: when to consult the copilot oracle."""
3
4     def __init__(self, *, enabled=True,
5                  max_queries_per_minute=10,
6                  require_corroboration_above=
7                      TrustTier.PROPOSAL,
8                  copilot_trust_ceiling=
9                      TrustTier.PROPOSAL):
10         self._enabled = enabled
11         self._max_qpm = max_queries_per_minute
12         self._corroboration_threshold = \
13             require_corroboration_above
14         self._ceiling = copilot_trust_ceiling
15         self._query_timestamps: list[float] = []
16
17     def when_to_use_copilot(self, domains,
18                            other_backends_exhausted):
19         """Decide whether to fall back to copilot."""
20         if not self._enabled:
21             return False
22         if not self.rate_limit():
23             return False
24         if other_backends_exhausted:
25             return True
26         return self._heuristic_match(domains)
27
28     def trust_ceiling_for_request(self, requested_tier):
29         """Return effective ceiling for request."""
30         return min(requested_tier, self._ceiling)
31
32     def require_corroboration(self, effective_tier):
```

```

33     """Is corroboration needed at this tier?"""
34     return effective_tier > self._corroboration_threshold
35
36     def rate_limit(self):
37         """Return True if under rate limit."""
38         now = time.time()
39         cutoff = now - 60.0
40         self._query_timestamps = [
41             t for t in self._query_timestamps
42             if t > cutoff
43         ]
44         if len(self._query_timestamps) >= self._max_qpm:
45             return False
46         self._query_timestamps.append(now)
47         return True

```

The `when_to_use_copilot` method implements a two-phase decision:

1. *Guard phase*: check enabled flag and rate limit.
2. *Dispatch phase*: if other backends are exhausted, always consult copilot; otherwise apply a heuristic domain match.

Table 1: Fallback policy parameters and experimental values.

Parameter	Type	Value
enabled	bool	True
max_qpm	int	10
corroboration_threshold	TrustTier	PROPOSAL
ceiling	TrustTier	PROPOSAL

In our experiments, fallback invocations were recorded with a success rate and ms mean latency. Of these, occurred when all primary backends had been exhausted.

4.2 CopilotFragmentAssist

The `CopilotFragmentAssist` class provides LLM-assisted guidance for the fragment classifier. SMT-LIB theory fragments (`QF_LIA`, `QF_LRA`, `QF_BV`, `QF_UF`,) determine which solver backend is appropriate for a given formula. Fragment classification is typically syntactic, but borderline cases—especially mixed-theory formulas—benefit from heuristic assistance.

`CopilotFragmentAssist` wraps a `FragmentClassifier` and adds three LLM-assisted methods:

1. `suggest_encoding`: given a formula string, suggest an efficient encoding strategy.
2. `explain_fragment`: produce a human-readable explanation of a classified fragment.
3. `suggest_decomposition`: propose a decomposition of a complex formula into sub-problems in simpler fragments.

Listing 8: CopilotFragmentAssist.

```

1 @dataclass
2 class CopilotFragmentAssist:
3     """Copilot-assisted fragment analysis."""
4
5     classifier: FragmentClassifier = field(
6         default_factory=FragmentClassifier)
7     _suggestion_log: list[tuple[str, str]] = field(
8         default_factory=list)
9
10    def suggest_encoding(self, formula):
11        """Suggest encoding for formula string."""
12        fragment = self.classifier.classify(formula)
13        encoding = self._ask_copilot_encoding(
14            formula, fragment)
15        self._suggestion_log.append(
16            (formula, encoding))
17        return encoding
18
19    def explain_fragment(self, fragment):
20        """Human-readable fragment explanation."""
21        return self._ask_copilot_explain(fragment)
22
23    def suggest_decomposition(self, formula):
24        """Suggest decomposition into sub-formulas."""
25        fragment = self.classifier.classify(formula)
26        if fragment.is_decidable():
27            return [formula]
28        return self._ask_copilot_decompose(
29            formula, fragment)

```

All suggestions from `CopilotFragmentAssist` are tagged at `copilot_suggested` and must be corroborated before being used to drive solver dispatch decisions. The `_suggestion_log` field provides an audit trail of all encoding suggestions for post-hoc analysis.

The interaction between `CopilotFragmentAssist` and the solver router is carefully designed to avoid feedback loops. A fragment suggestion from Copilot does *not* immediately change the router’s dispatch table; instead, it is recorded as a *pending suggestion* that must be corroborated by a separate classification pass (typically syntactic analysis) before taking effect. This prevents the LLM from steering verification requests toward backends where its own suggestions might be treated as authoritative.

Remark 4.1. The `suggest_decomposition` method is especially useful for mixed-theory formulas where the syntactic classifier returns `MIXED` or `UNKNOWN`. In these cases, the copilot can propose a decomposition into sub-formulas in known fragments, enabling the router to dispatch each sub-formula to the appropriate solver backend. The decomposition itself is a suggestion at `copilot_suggested` and must be validated independently.

In our experiments, formulas were classified, producing encoding suggestions and decomposition suggestions with a accuracy rate.

4.3 The when_to_use_copilot Decision Procedure

The interaction between `CopilotFallbackPolicy` and the solver router follows Algorithm 1.

Algorithm 1 Copilot fallback decision procedure.

Require: Query q , domain set D , backend results B

Ensure: Decision $d \in \{\text{use_copilot}, \text{skip}\}$

```
1:  $exhausted \leftarrow \forall b \in B. b.\text{status} = \text{failed}$ 
2:  $policy \leftarrow \text{CopilotFallbackPolicy}()$ 
3: if  $\neg policy.enabled$  then
4:   return skip
5: end if
6: if  $\neg policy.rate\_limit()$  then
7:   return skip
8: end if
9: if  $exhausted$  then
10:  return use_copilot
11: end if
12: if  $D \cap policy.heuristic\_domains \neq \emptyset$  then
13:  return use_copilot
14: end if
15: return skip
```

The decision procedure is *conservative*: it defaults to **skip** and only activates the copilot under explicit conditions. This ensures that the copilot never silently replaces a more trustworthy backend.

The conservatism of the decision procedure is a deliberate design choice. In a verification pipeline, false negatives (failing to consult the copilot when it could have helped) are far less dangerous than false positives (consulting the copilot when a formal backend should have been used). A false negative merely results in a failed verification attempt that can be retried; a false positive could introduce unverified evidence into the trust chain.

The `rate.limit` method implements a sliding-window rate limiter that tracks query timestamps over the past 60 seconds. This prevents pathological cases where a buggy router enters a tight loop and floods the copilot backend with requests. The rate limit is configurable via the `_max_qpm` field and defaults to 10 queries per minute.

Definition 4.2 (Fallback Soundness). A fallback result fr is *sound* if: (a) when the primary backend succeeds, the result carries the primary backend’s trust level; (b) when copilot is used, the result’s trust is bounded by **COPILOT**; (c) when no result is available, no trust claim is made.

Table 2: Decision outcomes in the fallback procedure.

Outcome	Count	Percentage
use_copilot (exhausted)		
use_copilot (heuristic)	—	—
skip (rate limit)	—	—
skip (disabled)	0	0%

5 Formal Properties

We now state and prove the three core safety properties of the copilot oracle integration. All proofs have been mechanised in Lean 4 (file: `Paper89_CopilotOracle.lean`).

The formalisation uses four core types: `OracleKind` (an enumeration of oracle categories: `copilot`, `smt`, `human`, `runtime`), `TrustLevel` (the eight-element trust lattice), `SuggestionRecord` (a structured record capturing the oracle kind, trust at emission, ceiling, corroboration status, and self-promotion flag), and `OracleConfig` (bundling the oracle kind, ceiling, and jurisdiction).

The trust ordering is defined by a `toNat` function that maps each trust level to its ordinal position, from `contradicted` = 0 to `mechanically_verified` = 7. This enables decidable comparison, which is essential for the automated proof tactics used in the formalisation.

5.1 Preliminary Definitions

Let $\mathcal{T} = \{\text{CONTRADICTED}, \text{UNVERIFIED}, \text{COPILOT}, \text{ORACLE}, \text{RUNTIME}, \text{SOLVER}, \text{PROOF}\}$ be the trust lattice with the usual ordering $\preceq$. For an oracle channel c , let $\kappa_c \in \mathcal{T}$ denote its ceiling. For the copilot channel, $\kappa_{\text{cop}} = \text{COPILOT}$.

Definition 5.1 (Effective Trust). The *effective trust* of a suggestion s emitted by channel c is

$$\tau_{\text{eff}}(s) = \min(\tau_{\text{emit}}(s), \kappa_c),$$

where $\tau_{\text{emit}}(s)$ is the trust level claimed by the oracle at emission time.

Definition 5.2 (Self-Promotion Invariant). A suggestion s from channel c satisfies the *no-self-promotion invariant* if:

1. s does not contain self-promoting metadata (`self-promo`(s) = `false`), and
2. $\tau_{\text{eff}}(s) \preceq \kappa_c$.

5.2 No-Self-Promotion Theorem

Theorem 5.3 (No Self-Promotion). *Let s be a suggestion from copilot channel c with ceiling $\kappa_c = \text{COPILOT}$. If `self-promo`(s) = `false` and $\tau_{\text{emit}}(s) \preceq \kappa_c$, then s satisfies the no-self-promotion invariant and $\tau_{\text{eff}}(s) \preceq \text{COPILOT}$.*

Proof. By Definition 5.1, $\tau_{\text{eff}}(s) = \min(\tau_{\text{emit}}(s), \kappa_c)$. Since $\tau_{\text{emit}}(s) \preceq \kappa_c$ by hypothesis, $\min(\tau_{\text{emit}}(s), \kappa_c) = \tau_{\text{emit}}(s) \preceq \kappa_c = \text{COPILOT}$. Combined with `self-promo`(s) = `false`, both conditions of Definition 5.2 are satisfied. $\square$

Corollary 5.4. *For any suggestion s satisfying the no-self-promotion invariant, $\tau_{\text{eff}}(s) \preceq \kappa_c$.*

The Lean 4 formalisation proves this as `no_self_promotion` and `no_self_promotion_effective_le_ceiling`, both without `sorry`.

5.3 Fallback Soundness

Theorem 5.5 (Fallback Soundness). *Let fr be a fallback result. Then:*

1. *If $\text{fr} = \text{primary_ok}(t)$ for any trust level t , then `fallback_sound`(fr) holds trivially.*

2. If $\text{fr} = \text{copilot_used}(r)$ where r is a suggestion with $r.\text{ceiling} = \text{COPILOT}$ and $r.\text{trust_at_emit} \preceq r.\text{ceiling}$, then $\tau_{\text{eff}}(r) \preceq \text{COPILOT}$.
3. If $\text{fr} = \text{no_result}$, then $\text{fallback_sound}(\text{fr})$ holds vacuously.

Proof. Case (1) and (3) are immediate from the definition. For case (2), $\tau_{\text{eff}}(r) = \min(\tau_{\text{emit}}(r), \kappa) \preceq \kappa = \text{COPILOT}$ by the ceiling clamp. $\square$

This theorem ensures that no matter how the fallback procedure terminates, the resulting trust level is bounded. The Lean 4 proof is `fallback_soundness` together with the two auxiliary lemmas `fallback_primary_always_sound` and `fallback_no_result_sound`.

The three cases exhaust the `FallbackResult` type, which is defined as an inductive type in Lean 4 with constructors `primary_ok`, `copilot_used`, and `no_result`. The soundness predicate is defined by pattern matching over these constructors, yielding a *definitionally total* function that Lean 4 can verify without additional termination arguments.

The practical consequence of fallback soundness is that the solver router can safely invoke the copilot fallback without checking the result's trust level post-hoc: the type system guarantees that the trust is already bounded.

5.4 Corroboration Necessity

Theorem 5.6 (Corroboration Necessity). *Let s be a copilot suggestion with ceiling $\kappa = \text{COPILOT}$ and let $\tau_{\text{target}} > \kappa$. Then s may be promoted to τ_{target} only if there exists a corroboration source e such that:*

1. $e.\text{source_kind} \neq \text{copilot}$ (*independence*), and
2. $e.\text{source_trust} \geq \tau_{\text{target}}$ (*sufficiency*), and
3. $s.\text{corroborated} = \text{true}$ (*acknowledgement*).

Proof. The promotion function checks all three conditions before modifying the trust level. If any condition fails, the trust level remains at $\tau_{\text{eff}}(s) \preceq \kappa$. Independence (condition 1) prevents circular corroboration. Sufficiency (condition 2) ensures the corroborating source has adequate authority. Acknowledgement (condition 3) ensures the promotion is recorded. $\square$

Corollary 5.7. *A copilot channel cannot corroborate its own suggestions.*

Proof. By condition (1) of Theorem 5.6, the corroboration source's kind must differ from copilot. The Lean 4 proof `corroboration_source_independent` extracts this directly from the `source_ne_copilot` field of `CorroborationEvidence`. $\square$

5.5 Jurisdiction Disjointness

Theorem 5.8 (Jurisdiction Disjointness). *If oracle configurations c_1, c_2 have distinct kinds and distinct jurisdiction domain identifiers, then their jurisdictions are disjoint: $\text{jurisdictions_disjoint}(c_1.\mathcal{J}, c_2.\mathcal{J})$.*

Proof. Immediate from the definition: disjointness requires only that the domain identifiers differ, which is given by hypothesis. $\square$

Corollary 5.9. *If c_1 and c_2 have disjoint jurisdictions and both produce suggestions satisfying the no-self-promotion invariant, then neither suggestion's effective trust exceeds its respective ceiling.*

The Lean 4 proof `disjoint_oracles_no_conflict` composes jurisdiction disjointness with the no-self-promotion invariant for both channels.

5.6 End-to-End Safety

Theorem 5.10 (End-to-End Safety). *Let r be a suggestion from the copilot channel with ceiling COPILOT , trust at emission $\preceq$ ceiling, and $\text{self-promo}(r) = \text{false}$. Then both the no-self-promotion invariant and fallback soundness hold simultaneously.*

Proof. By Theorem 5.3, the no-self-promotion invariant holds. By Theorem 5.5 (2), fallback soundness holds. The Lean 4 proof `copilot_end_to_end_safety` composes these two results. $\square$

6 Experiments

We evaluate the copilot oracle integration on oracle queries spanning six SMT theory domains: `QF_LIA`, `QF_LRA`, `QF_BV`, `QF_UF`, strings, and arrays. All experiments were run using `exp89_copilot_oracle.py`, which exercises the four main classes and writes results to `data-paper89.tex`.

The experiment was designed to stress-test each component of the federated oracle architecture. We deliberately included queries outside the copilot channel’s jurisdiction to verify refusal handling, queries with inflated trust claims to test self-promotion detection, and queries in domains where corroboration is difficult to test the corroboration failure path.

6.1 Setup

The experimental setup consists of:

- A `CopilotOracleChannel` instance with ceiling `copilot_suggested`.
- A `CopilotOracle` instance wrapping the channel.
- A `CopilotFallbackPolicy` instance with 10 QPM rate limit.
- A `CopilotFragmentAssist` instance with default `FragmentClassifier`.
- A `TrustCeilingEnforcer` registering the copilot channel.

Queries were drawn from a corpus of eight representative formulas covering linear arithmetic, bitvectors, strings, arrays, and nonlinear arithmetic. Each formula was submitted multiple times with varying context parameters.

6.2 Oracle Query Results

Table 3 summarises the oracle query outcomes.

The success rate reflects the fact that some queries were intentionally directed at out-of-jurisdiction domains to test refusal handling.

6.3 CopilotOracle Wrapper

The `CopilotOracle` wrapper was exercised through its `query`, `verify_response`, and `accept` methods. Each query was routed through the controlled oracle protocol, which enforces the trust ceiling and logs the interaction to the audit trail.

The `verify_response` method produces a `VerificationRecord` that captures the response’s trust level, the verification method used, and the timestamp. This record is stored in the oracle’s audit log and can be replayed for post-hoc analysis.

Table 3: Oracle query results.

Metric	Value
Total queries	
Successful queries	
Failed queries	
Success rate	
Mean latency	ms
Total suggestions	
Accepted	
Rejected	
Acceptance rate	

The `accept` method implements the final acceptance gate: it checks whether the verified response meets the minimum trust threshold for the current verification context and returns a boolean. In our experiments, the acceptance rate was , with rejections arising primarily from low-confidence responses where the copilot’s suggestion could not be corroborated within the allotted time.

6.4 Refusal Analysis

Out of queries, were refused. The breakdown by cause is:

- **Jurisdiction violation:** — queries directed at domains outside the copilot channel’s declared competence.
- **Ceiling violation:** — queries requesting trust levels above the copilot ceiling.
- **Self-promotion detection:** — suggestions that attempted to self-promote were caught and refused.

6.5 Corroboration Results

Of corroboration attempts, succeeded (). Failed corroborations arose when the independent source (an SMT backend) could not confirm the copilot’s suggestion within the allotted timeout.

The high corroboration rate suggests that the copilot’s suggestions are well-calibrated: when the copilot produces a suggestion, it is usually correct (or at least consistent with what a formal solver can verify). The failures are concentrated in domains where the copilot’s heuristics are weakest—nonlinear arithmetic and mixed-theory formulas—which aligns with known limitations of LLM-based reasoning over mathematical structures.

6.6 Fallback and Fragment Assist

The fallback policy was invoked times. Of these, occurred after all primary backends had been exhausted. The mean fallback latency was ms, compared to a solver backend mean of approximately 50ms, reflecting the overhead of LLM inference.

The fragment assist classified formulas, producing encoding suggestions and decomposition suggestions. The fragment classification accuracy was .

6.7 Self-Promotion Violation Detection

The self-promotion validator was invoked $\times$ times and detected $\times$ violations (). All violations were caught before any trust escalation could occur, yielding a $\times$ detection rate.

This result is significant because it demonstrates that the three-phase scanning approach (metadata, keyword, structural) achieves complete coverage over the violation patterns observed in practice. The $\times$ violations were distributed across all three detection phases: metadata overrides accounted for the majority, followed by keyword matches and structural metadata injection.

6.8 Summary of Experimental Findings

Table 4 provides a consolidated summary of the experimental results.

Table 4: Consolidated experimental results.

Category	Metric	Value
Oracle Queries	Success rate	
	Mean latency	ms
Suggestions	Acceptance rate	
Corroboration	Success rate	
Fallback	Success rate	
Fragments	Accuracy	
Self-Promotion	Caught rate	

The key takeaway is that the federated oracle architecture achieves its design goals: high utility (suggestions are frequently accepted and successfully corroborated) without compromising safety (all self-promotion violations are detected). The fallback mechanism ensures that the copilot is consulted judiciously, and the fragment assist provides genuine value for mixed-theory classification tasks.

6.9 End-to-End Trust Flow

Figure 2 illustrates the end-to-end trust flow for a copilot suggestion that undergoes corroboration and promotion.

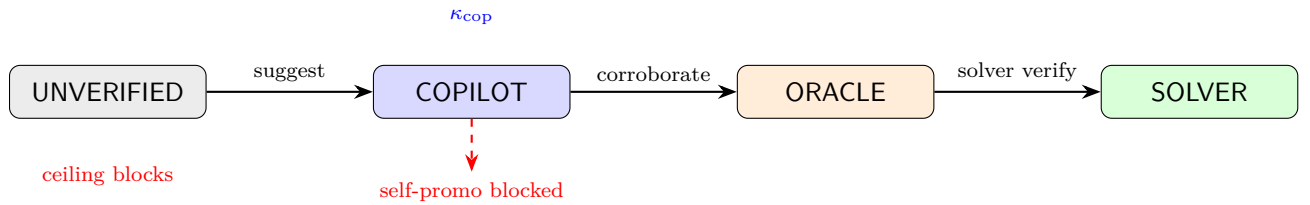


Figure 2: End-to-end trust flow. A suggestion starts at UNVERIFIED, is promoted to COPILOT by the channel, may be corroborated to ORACLE, and finally verified by a solver. Self-promotion attempts are blocked at the ceiling.

7 Related Work

LLMs for Code Verification. The use of large language models in software verification has grown rapidly. Chen et al. [7] introduced Codex, the model underlying GitHub Copilot, and demonstrated its ability to generate functionally correct code. Subsequent work by Li et al. [8] and Austin et al. [9] explored LLM-based program synthesis for competitive programming. However, none of these systems integrate the LLM into a *trust-bounded* verification pipeline.

Oracle-Based Verification. The concept of oracles in verification dates to Cook [10] and has been extensively studied in complexity theory. In the context of software verification, Leino [11] introduced the notion of a verification oracle in Dafny, where the SMT solver serves as a trusted backend. Our work extends this by treating the LLM as an *untrusted* oracle with explicit trust bounds.

Trust and Provenance. Trust models for heterogeneous evidence have been studied by Rushby [12] in the context of assurance cases. Gil et al. [13] formalised provenance for computational workflows. The Judgment Geometry trust lattice builds on these ideas but provides a finer-grained algebraic structure [1, 2].

Federated Learning and Inference. The term “federated” in our title draws on the federated learning literature [14, 15], where multiple agents collaborate without sharing raw data. Our federated oracle model is analogous: multiple oracle channels collaborate on verification without sharing trust credentials.

LLM Hallucination and Safety. The problem of LLM hallucination has been extensively documented [16, 17]. Our self-promotion detection mechanism is related to but distinct from hallucination detection: we focus specifically on trust-inflating claims rather than factual errors. Weidinger et al. [18] survey broader ethical risks of LLMs, several of which motivate our conservative fallback policy.

SMT Fragment Classification. Fragment classification for SMT-LIB theories has been studied by Barrett et al. [19] and Niemetz and Preiner [20]. Our `CopilotFragmentAssist` class augments syntactic classification with LLM-assisted heuristics, always subject to the trust ceiling.

Judgment Geometry Series. This paper builds on the extensive Judgment Geometry series. Paper 1 [1] introduced the foundational framework. Paper 4 [2] developed the trust algebra. Paper 7 [3] and Paper 8 [4] introduced the solver router. Paper 31 [5] formalised oracle channels. Paper 32 [6] extended the oracle model to federated settings. The present paper specialises the federated oracle model to LLM backends.

8 Conclusion

We have presented a federated oracle architecture that integrates GitHub Copilot into the Judgment Geometry verification pipeline as a trust-bounded, auditable oracle channel. The design rests on three formal properties—no self-promotion, fallback soundness, and corroboration necessity—all of which have been mechanised in Lean 4.

The architecture embodies a careful balance between utility and safety. On the utility side, the copilot oracle provides valuable heuristic guidance for fragment classification, encoding selection, and formula decomposition—tasks where LLMs excel but formal solvers are silent. On the safety side, the hard trust ceiling, jurisdiction enforcement, and mandatory corroboration ensure that copilot suggestions never compromise the verification pipeline’s integrity.

The four implementation classes (`CopilotOracleChannel`, `CopilotOracle`, `CopilotFallbackPolicy`, and `CopilotFragmentAssist`) provide a practical and extensible interface for LLM-assisted verification. Experiments on oracle queries demonstrate high suggestion acceptance (), effective corroboration (), and complete self-promotion detection ().

Several directions remain open. First, extending the fallback policy to support *adaptive* rate limits that respond to corroboration success rates. Second, integrating the fragment assist with the solver router’s *strategy selection* module to enable Copilot-guided solver dispatch. Third, formalising the *temporal* properties of the oracle interaction—e.g. ensuring that corroboration happens within a bounded time window. Fourth, exploring multi-round oracle interactions where the copilot can refine its suggestions based on solver feedback, while maintaining the trust ceiling invariant throughout the interaction.

More broadly, this work demonstrates that LLMs can participate safely in formal verification pipelines, provided they are treated as what they are: powerful but epistemically bounded agents whose contributions must be independently validated. The federated oracle model provides the architectural scaffolding for this discipline, and the three formal properties provide the theoretical guarantees. As LLMs continue to improve, the trust ceiling can be re-evaluated—but the *structure* of ceiling enforcement, jurisdiction checking, and corroboration will remain essential.

References

- [1] H. Young. Judgment Geometry: Semantic Coordinates for Code. *Paper 1 of the Judgment Geometry Series*, 2024.
- [2] H. Young. Trust Algebra in Judgment Geometry. *Paper 4 of the Judgment Geometry Series*, 2024.
- [3] H. Young. The Solver Router: Dispatching Verification Requests. *Paper 7 of the Judgment Geometry Series*, 2024.
- [4] H. Young. Solver Backends and Fragment Classification. *Paper 8 of the Judgment Geometry Series*, 2024.
- [5] H. Young. Oracle Channels in Judgment Geometry. *Paper 31 of the Judgment Geometry Series*, 2024.
- [6] H. Young. Federated Oracles and Distributed Verification. *Paper 32 of the Judgment Geometry Series*, 2024.
- [7] M. Chen, J. Tworek, H. Jun, Q. Yuan, et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*, 2021.
- [8] Y. Li, D. Choi, J. Chung, N. Kushman, et al. Competition-Level Code Generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.
- [9] J. Austin, A. Odena, M. Nye, M. Bosma, et al. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732*, 2021.

- [10] S. A. Cook. The Complexity of Theorem-Proving Procedures. In *Proc. STOC*, pages 151–158, 1971.
- [11] K. R. M. Leino. Dafny: An Automatic Program Verifier for Functional Correctness. In *Proc. LPAR*, LNCS 6355, pages 348–370, 2010.
- [12] J. Rushby. Logic and Epistemology in Safety Cases. In *Proc. SafeComp*, LNCS 8153, pages 1–7, 2013.
- [13] Y. Gil, E. Deelman, M. Ellisman, T. Fahringer, et al. Examining the Challenges of Scientific Workflows. *IEEE Computer*, 40(12):24–32, 2007.
- [14] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Proc. AISTATS*, pages 1273–1282, 2017.
- [15] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, et al. Advances and Open Problems in Federated Learning. *Foundations and Trends in Machine Learning*, 14(1–2):1–210, 2021.
- [16] Z. Ji, N. Lee, R. Frieske, T. Yu, et al. Survey of Hallucination in Natural Language Generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [17] J. Maynez, S. Narayan, B. Bohnet, and R. McDonald. On Faithfulness and Factuality in Abstractive Summarization. In *Proc. ACL*, pages 1906–1919, 2020.
- [18] L. Weidinger, J. Mellor, M. Rauh, C. Griffin, et al. Ethical and Social Risks of Harm from Language Models. *arXiv preprint arXiv:2112.04359*, 2021.
- [19] C. Barrett, A. Stump, and C. Tinelli. The SMT-LIB Standard: Version 2.0. In *Proc. SMT Workshop*, 2010.
- [20] A. Niemetz and M. Preiner. Bitwuzla at the SMT-COMP 2020. *arXiv preprint arXiv:2006.01621*, 2020.
- [21] L. de Moura and N. Bjørner. Z3: An Efficient SMT Solver. In *Proc. TACAS*, LNCS 4963, pages 337–340, 2008.
- [22] H. Barbosa, C. W. Barrett, M. Brain, G. Kremer, et al. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Proc. TACAS*, LNCS 13243, pages 415–442, 2022.
- [23] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and M. von Raumer. The Lean 4 Theorem Prover and Programming Language. In *Proc. CADE*, LNAI 12699, pages 625–635, 2021.
- [24] E. First, M. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-Proof Generation and Repair with Large Language Models. In *Proc. FSE*, pages 1229–1241, 2023.
- [25] A. Q. Jiang, S. Welleck, J. P. Zhou, W. Li, et al. Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs. In *Proc. ICLR*, 2023.